

Vertica Analytics Platform

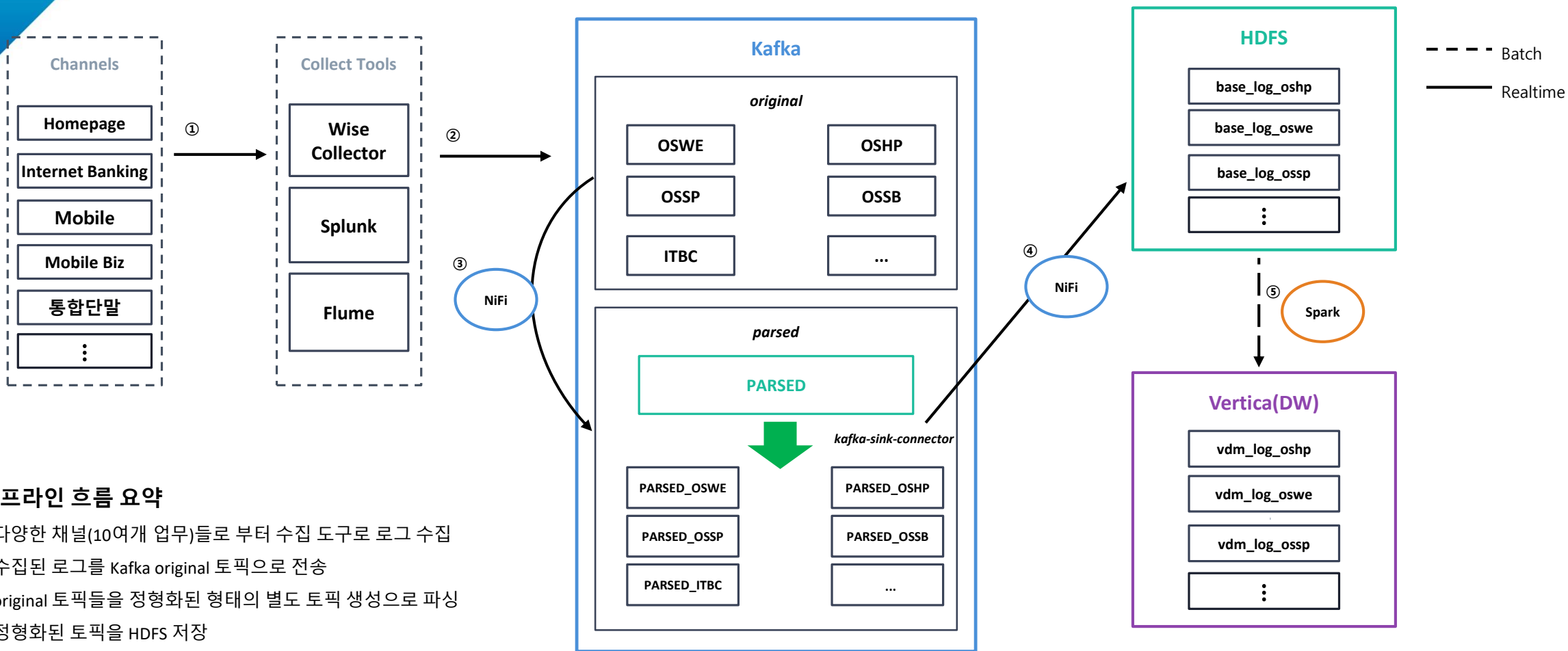
Kafka와 Vertica를 활용한 금융권 데이터 파이프라인 활용 사례



S은행 사례

. 01

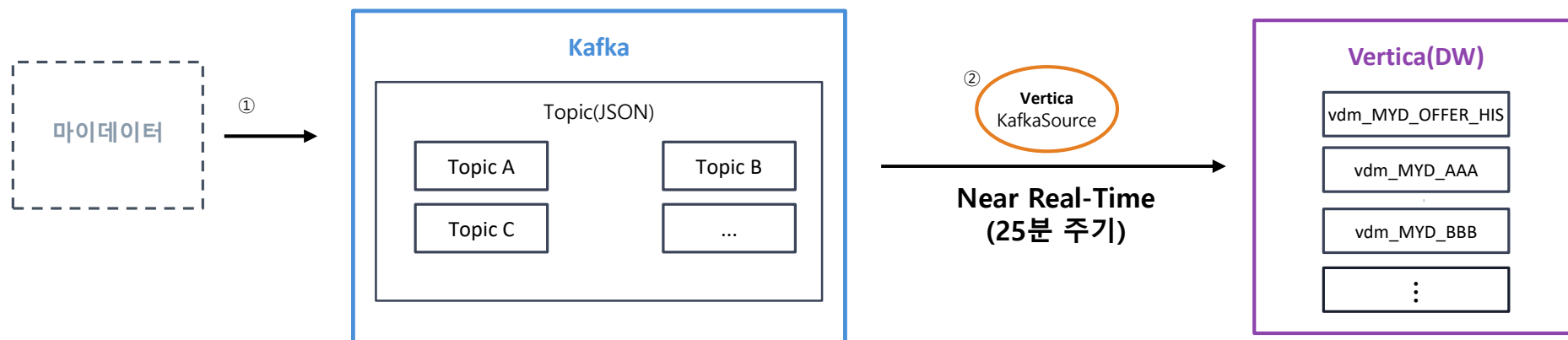
고객행동 로그 분석 사례



파이프라인 흐름 요약

- ① 다양한 채널(10여개 업무)들로부터 수집 도구로 로그 수집
- ② 수집된 로그를 Kafka original 토픽으로 전송
- ③ original 토픽들을 정형화된 형태의 별도 토픽 생성으로 파싱
- ④ 정형화된 토픽을 HDFS 저장
- ⑤ Spark을 활용해서 Vertica에 적재

마이데이터의 정보제공 이력 활용 사례



Vertica Kafka Consume 기능

COPY raw_iot

```
SOURCE KafkaSource(stream='iot-data|0|-2,iot-data|1|-2,iot-data|2|-2', brokers='docd01:6667,docd03:6667', stop_on_eof=TRUE)
PARSER KafkaParser();
```

COPY logs

```
SOURCE KafkaSource(stream='server_log|0|0', stop_on_eof=true, duration=interval '10 seconds', brokers='kafka01:9092')
PARSER KafkaJSONParser(flatten_arrays=True, flatten_maps=True);
```

COPY weather_logs

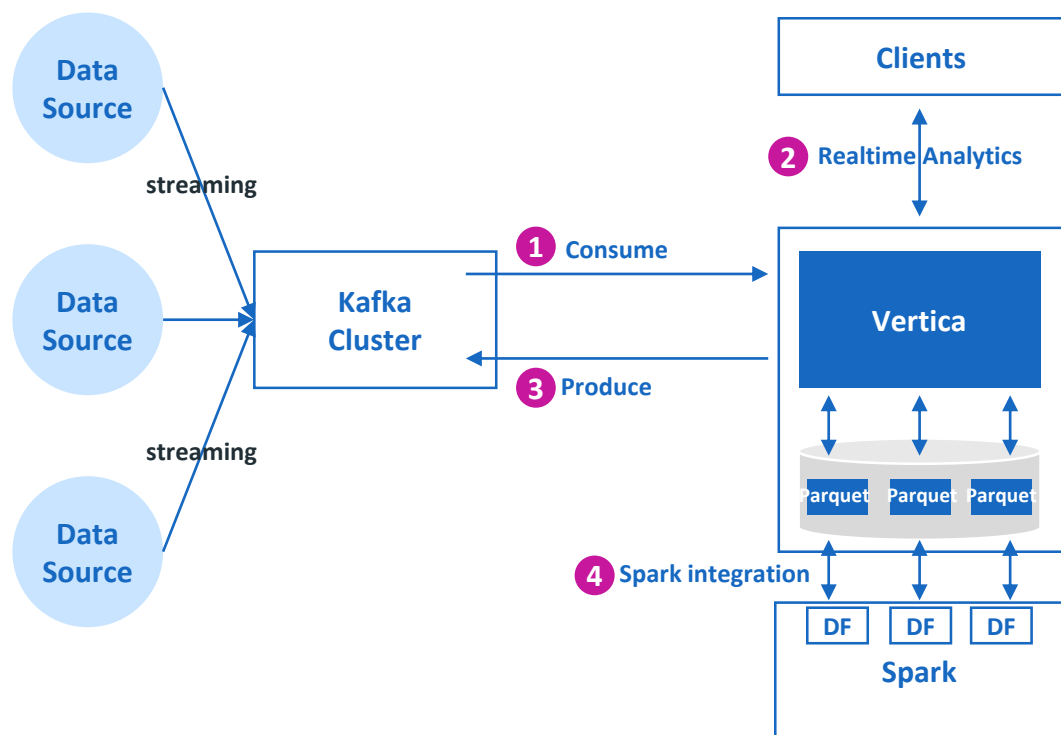
```
SOURCE KafkaSource(stream='temperature|0|0', brokers='kafka01:9092, kafka01:9093', stop_on_eof=true, duration=interval '10 seconds')
PARSER KafkaAvroParser(flatten_arrays=False, flatten_maps=False, flatten_records=False,
생략
);
```

파이프라인 흐름 요약

- ① 마이데이터 정보 제공 이력등에 대한 정보를 Kafka 토픽으로 전송
- ② Vertica에서 직접 Kafka 토픽을 Consume 하여 데이터 분석에 활용(25분 주기로 수집 중)

Kafka + Vertica 연계

다양한 채널 및 플랫폼에서 유입되는 데이터를 실시간으로 저장/분석하기 위해서는 데이터 파이프 라인 플랫폼과의 유기적인 연계가 필수적입니다.



1 Consume

별도의 솔루션이나 커넥터 없이 카프카 토픽을 적재하는 기능을 빌트인 함수로 제공
카프카 토픽의 파티션을 병렬로 적재하여 실시간 스트리밍 데이터 처리에 최적화 되어 있으며,
자동으로 지속적인 컨슈밍을 지원하기 위한 마이크로배치 기능 탑재

2 Realtime Analytics

매우 짧은 주기로 적재되는 대용량 데이터를 버티카를 통해 분석 수행

3 Produce

버티카에 저장된 데이터를 카프카로 추출하는 기능 제공
추가적인 구축 없이 데이터 파이프 라인과 양방향으로 데이터를 주고 받을 수 있는 아키텍처

4 Spark integration

Spark 연계를 위한 커넥터가 기본 제공
Spark에서 분석이 필요한 업무를 위해 빠르게 스파크 메모리로 버티카 데이터를 로드하여 실시간 분석 수행
오브젝트 스토리지를 스테이징 파일 시스템으로 활용하여 병렬 추출 및 병렬 적재 지원
스파크에서 분석된 결과도 버티카로 병렬 추출 및 적재 수행

Vertica

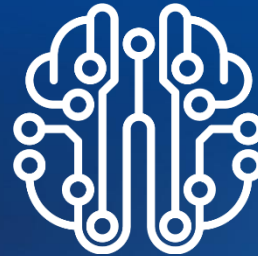
. **02**

What is VERTICA?

SQL Data Warehouse



Analytics & Machine Learning



Query Engine

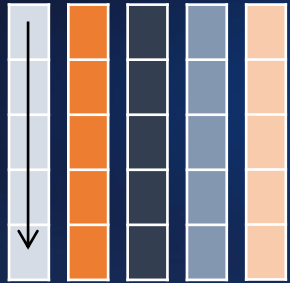


- MPP + Columnar 아키텍처를 통해 **대용량 데이터 분석**에 최적화
- On-premise / Public Cloud / Private Cloud / k8s 등 **어떤 인프라 환경**에서도 배포 및 운영 지원
- **I/O 최적화 아키텍처**로 전통적인 DW 환경보다 10-50배 이상 빠른 처리 성능 제공

- **Time series / Full text search** 등의 **고급 분석 기능** 지원
- **페타 바이트 규모**의 In-Database Machine Learning 지원
- 사용자에게 친숙한 **SQL로 ML 프로세스** 지원

- **HDFS / Object Store**에 저장된 다양한 포맷(Parquet/ORC/Iceberg 등) 데이터를 직접 조회
- DW 데이터와 Data Lake에 존재하는 **데이터 연계 분석**(Join 처리) 지원
- **Complex Type**을 활용 (ARRAY / MAP / STRUCT 등)
- **Vector data 저장 및 분석 함수** 지원

VERTICA 주요 기술 요소



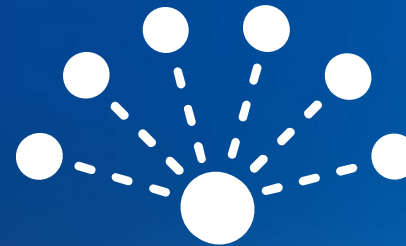
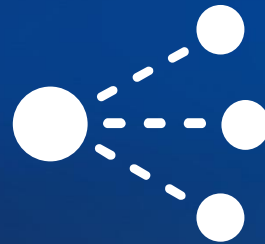
Native
Columnar Storage

- 필요한 컬럼만을
조회하여 빠른 쿼리
성능 보장



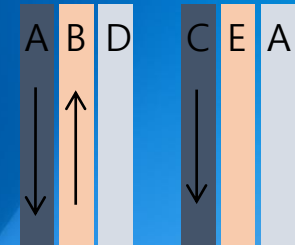
Compression
/Encoding

- 저장공간과
I/O의 효율성
을 위한
압축과 인코딩



MPP Scale-out + Distributed Query

- 대규모 병렬 처리 MPP 아키텍처
(Massively Parallel Processing)
- 분산 쿼리 수행을 통한 빠른 처리 성능



Projections

- 데이터 저장 시
분산/정렬 하여
저장 하므로
인덱스 없이도
쿼리 속도 극대화

Embracing an Open Source Architecture

Apache Spark,
Hadoop and Kafka
Integration



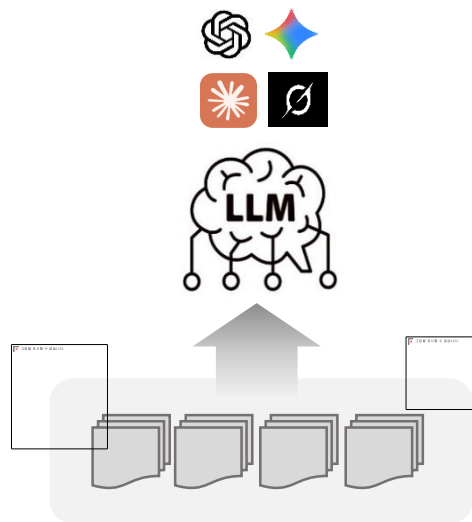
AI(LLM) + MCP + Vertica Text to SQL

. **03**

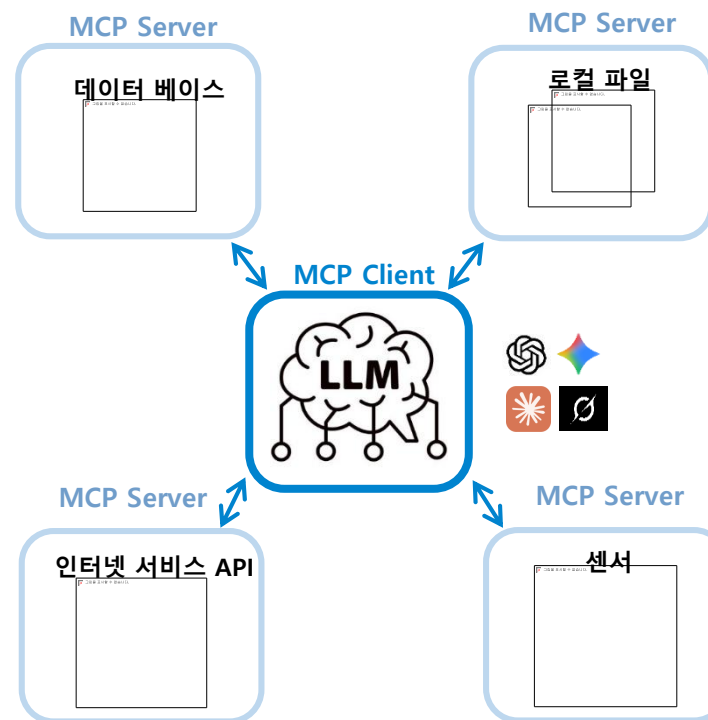
LLM 과 MCP

LLM은 학습 데이터에 의존적이므로 외부 시스템과의 통합에 한계가 있어
실제 비즈니스 환경에서 자율적으로 작동하는 AI Agent를 구축하기 위한 표준화된 프레임워크가 필요했습니다.
이러한 배경에서 MCP(Model Context Protocol)가 출현하게 되었습니다.

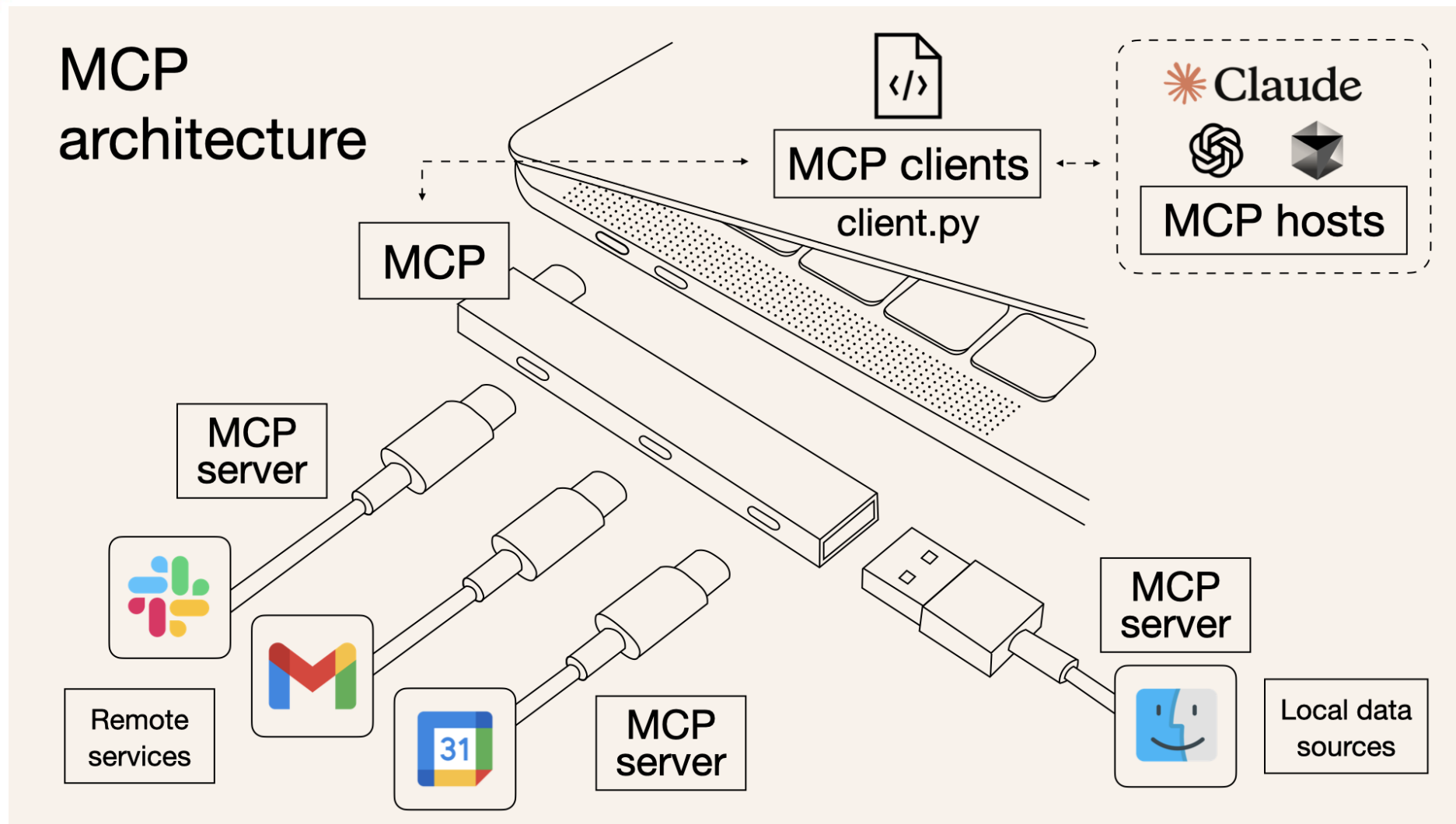
LLM > 학습 데이터 의존적



LLM + MCP > 더욱 다양한 데이터 활용 가능

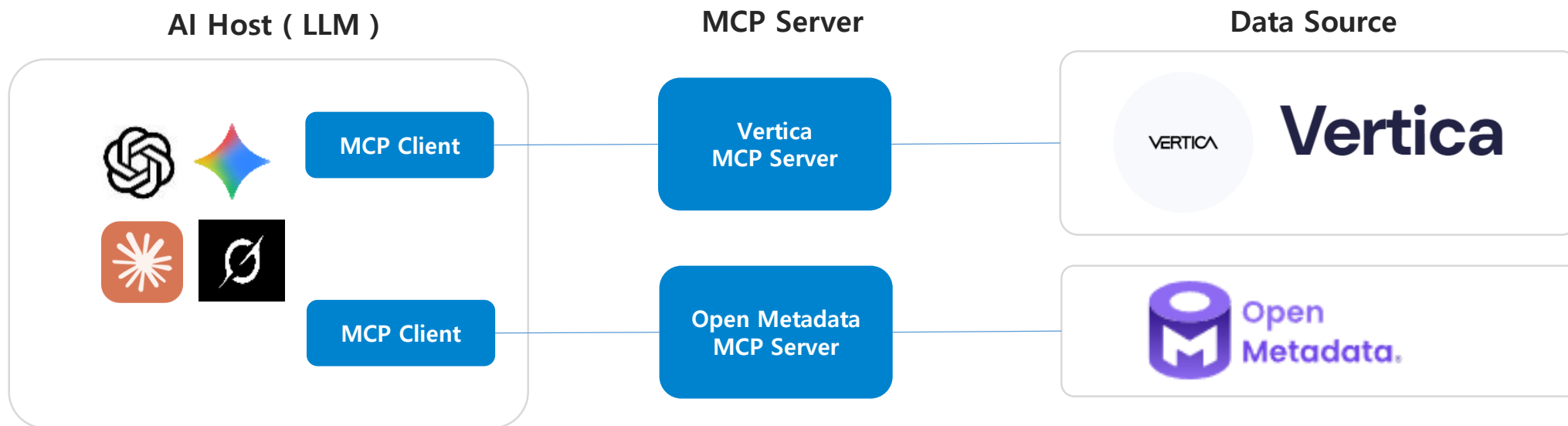


MCP(Model Context Protocol) Architecture

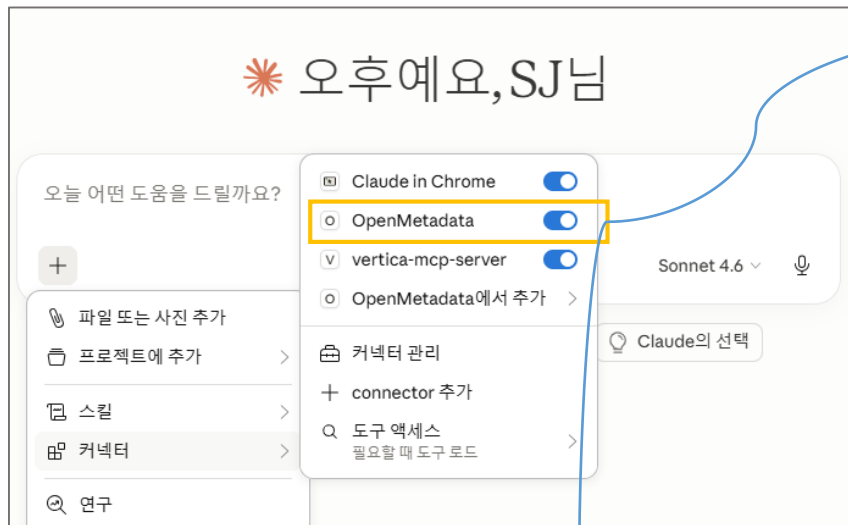


LLM + Vertica MCP 구성

- 자연어 기반에 데이터 조회(Text to SQL)를 위해서 AI와 Vertica 연동



Text to SQL (Vertica + MCP + Claude Desktop)



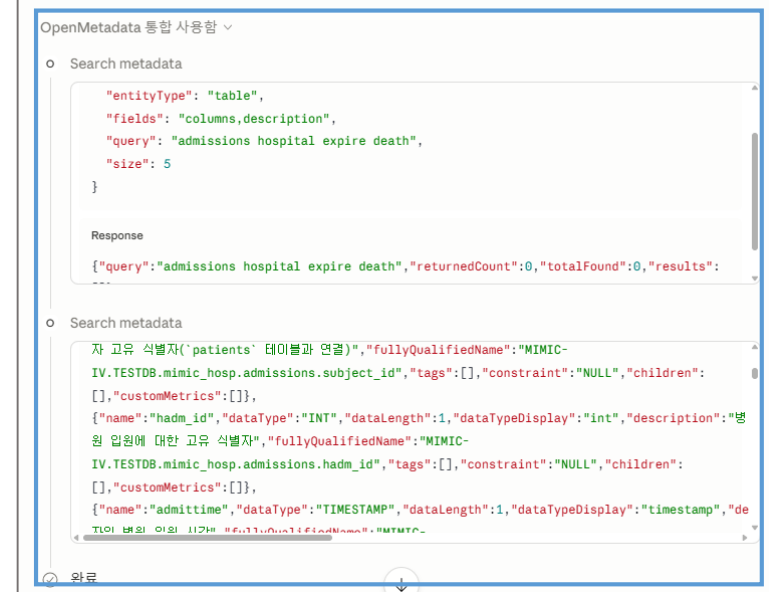
ICU 체류 기간 및 사망률 상관관계
-- 중환자실 체류 시간(LOS)과 사망 여부가 기록된 내용을 조회할거야.
-- 체류 기간을 2일 미만부터 3개월 이상까지 5단계로 나누고, 각 단계별 환자 수와 사망률을 분석해줘.
-- 체류 기간이 늘어남에 따라 사망 위험도가 어떻게 변하는지 설명해줘.

먼저 OpenMetadata와 Vertica에서 관련 테이블과 컬럼 정보를 파악하겠습니다.

OpenMetadata TOOL을 선택하여 1차적으로 요건에 맞는 메타데이터 정보 확인



이제 사망 정보가 있는 admissions 테이블도 확인합니다.



Text to SQL (Vertica + MCP + Claude Desktop)

※ 오후예요, SJ님

오늘 어떤 도움을 드릴까요?

+

파일 또는 사진 추가

프로젝트에 추가

스킬

커넥터

연구

Claude In Chrome

OpenMetadata

vertica-mcp-server

OpenMetadata에서 추가

커넥터 관리

connector 추가

도구 액세스

필요할 때 도구 로드

메타데이터 파악 완료. 이제 Vertica 쿼리를 실행합니다.

vertica-mcp-server 통합 사용함, 로드된 도구

Loading tools

Request
{
 "query": "vertica execute query SQL"
}

Response
Loaded 5 tools:

Submit query job

결과

메타데이터 확인이 완료 되면 VERTICA-MCP TOOL을 이용하여 데이터를 분석, 분석 결과 SQL, 시각화를 포함한 분석 결과 등의 제시 가능

Loading tools

Request
{
 "query": "vertica get job results status"
}

Response
Loaded 5 vertica-mcp-server tools:

Get job status

결과

완료

결과가 완료되었습니다. 이제 시각화를 포함한 분석 결과를 제시합니다.

총 ICU 입실 건수

94,458

전체 사망 건수

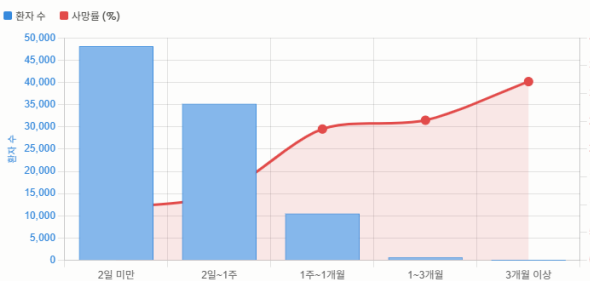
11,350

전체 사망률

12.02%

최대 사망률 구간

3개월 이상



체류 기간 구간	평균 LOS(일)	환자 수	사망 건수	사망률	사망률 시각화
1단계: 2일 미만	1.14	48,107	4,512	9.38%	
2단계: 2일 ~ 1주 미만	3.58	35,166	4,184	11.90%	
3단계: 1주 ~ 1개월 미만	12.69	10,498	2,479	23.61%	
4단계: 1개월 ~ 3개월 미만	41.27	659	166	25.19%	
5단계: 3개월 이상	122.81	28	9	32.14%	

분석 결과 요약

총 94,458건의 ICU 입실 기록에서 아래의 패턴이 명확하게 확인됩니다.

체류 기간이 길어질수록 사망률은 단조 증가합니다. 2일 미만(9.38%) → 2일~1주(11.90%) → 1주~1개월(23.61%) → 1~3개월(25.19%) → 3개월 이상(32.14%) 순으로 상승하며, 특히 1주를 넘어서는 시점에서 사망률이 약 2배 가까이 급등(11.90% → 23.61%)하는 점이 가장 두드러집니다.

임상적 해석:

- 1단계(2일 미만, 48,107명): 전체 환자의 약 51%가 집중되는 최대 구간입니다. 단기 회복 사례가 대부분이지만 9.38%의 사망률이 관찰되는데, 이는 입실 직후 급성기 사망이 포함된 결과입니다.
- 2단계(2일~1주, 35,166명): 단기~중기 치료 구간으로 사망률이 소폭 상승(11.90%)합니다. 상태가 안정화되지 못한 환자군이 여기에 해당합니다.
- 3단계(1주~1개월, 10,498명): 사망률이 23.61%로 급격히 상승하는 변곡점입니다. 장기 기계환기, 패혈증, 다발성 장기부전 환자가 집중되는 구간으로 해석됩니다.
- 4단계(1개월~3개월, 659명): 극소수 환자만 이 구간까지 생존하며, 사망률 25.19%로 3단계와 큰 차이를 보이지 않습니다. 만성 critical illness 환자들의 고원 구간으로 볼 수 있습니다.
- 5단계(3개월 이상, 28명): 사례 수가 극히 적어 통계적 해석에 주의가 필요하지만, 32.14%로 가장 높은 사망률을 보입니다. 장기 ICU 생존자 중에서도 회복 한계에 도달한 극중증 환자들입니다.

핵심 인사이트: 1주 미만 vs. 1주 이상의 경계가 임상적으로 가장 의미있는 임계점(threshold)이며, 이 지점을 넘는 환자에 대한 집중적인 다학제 개입(early mobilization, palliative care 연계 등)이 사망률 감소에 기여할 수 있습니다.

실행 SQL

```
sql
SELECT
  los_group,
  los_range,
  total_patients,
  death_count,
  ROUND(death_count * 100.0 / total_patients, 2) AS mortality_rate_pct,
  ROUND(avg_los_days, 2) AS avg_los_days
FROM (
  SELECT
    CASE
      WHEN i.los < 2 THEN 1
      WHEN i.los < 7 THEN 2
      WHEN i.los < 30 THEN 3
      WHEN i.los < 90 THEN 4
      ELSE 5
    END AS los_group,
    CASE
      WHEN i.los < 2 THEN '1단계: 2일 미만'
      WHEN i.los < 7 THEN '2단계: 2일 이상 ~ 1주 미만'
      WHEN i.los < 30 THEN '3단계: 1주 이상 ~ 1개월 미만'
      WHEN i.los < 90 THEN '4단계: 1개월 이상 ~ 3개월 미만'
      ELSE '5단계: 3개월 이상'
    END AS los_range,
    COUNT(*) AS total_patients,
    SUM(a.hospital_expire_flag) AS death_count,
    AVG(i.los) AS avg_los_days
  FROM mimiciu.icustays i
  JOIN mimichosp.admissions a
    ON i.hadm_id = a.hadm_id
  GROUP BY 1, 2
) t
ORDER BY los_group
```

사용 테이블:

- mimiciu.icustays — ICU 체류 기록 (los: 재실 기간(일), FLOAT)
- mimichosp.admissions — 입퇴원 정보 (hospital_expire_flag: 병원 내 사망 여부, 1=사망/0=생존)
- 조인 키: hadm_id (병원 입원 고유 식별자)

감사합니다

VERTICA